TLog Procesrapport

Thomas Erlang Sloth

2013

Contents

1 Forord **4**

2 Case **5**

3 Case analyse **7**

3.1 Funktionalitet 7

3.2 Stabilitet 7

3.3 Dokumentation 7

4 Problemformulering **8**

5 Estimeret tidsplan **9**

6 Metode og teknologivalg **9**

7 Topologi **10**

8 Log format **11**

9 Database **12**

9.1 Layout 13

9.2 Skalering 15

9.2.1 Lodret skalering 15

9.2.2 Vandret skalering 15

10 Beskedmodtagelse **17**

10.1 Besked parsing 17

10.2 Gruppering 18

10.3 Stikprøver 18

10.4 Filtrering 18

10.5 Skalering 20

11 Team **21**

12 Notifikationer **22**

13 Adaptivitet **23**

13.1 Early warning 24

14 Søgning **25**

15 Interface **25**

16 Watchdog **27**

17 Test **28**

18 Realiseret tidsplan **28**

19 Logbog **29**

20 Konklusion **32**

21 Perspektivering **34**

1 Forord

Denne rapport giver indblik i mine tanker omkring udviklingen af TLog.

2 Case

Tema/titel for	***SYSLOG protokol analysator (SPA)***	
Godkendt nummer	04102012-001	
Casebeskrivelsen	Syslog protokollen beskrevet i RFC5424 er en standard for logning af data. Syslog anvendes til system drift og vedligehold af blandede miljøer ved auditering af sikkerhed til simpel information og debugging. Herved generer selv et mindre system en enorm mængde af data – hvoraf langt hovedparten er irrelevante. Der ønskes udarbejdet løsning til håndtering af SYSLOG data i et skalerbart heterogent server/netværks/applikation-system. Via en syslog fortolker skal man, via et brugerinterface,kunne filtrere de viste syslog data efter de i RFC standarden angivne typer. Hermed skabes mulighed for et tydeligt overblik over hele systemet aktuelle tilstand – og skabe basis for en 'early warning' gennem en adaptivitet i filtreringsprocessen. Løsningen til SPA´en må ikke et indholde single point of failure, men det vægtes på trods heraf, at løsningen realiseres på en prisbillig hardware platform. Løsningen skal være skalerbar i området 'small business' til 'Campus area'.	

[1]

[1] https://dl.dropboxusercontent.com/u/91435/tlog/case_1.png

Fagområder som projektet dækker	Database	X
	Serverteknologi - Databaseserver	X
	Programmering – Java, Php eller C# (valgfri)	X
		☐
		☐
		☐
		☐
		☐
Valgfri specialefag som projektet dækker		

Bemærkninger.
Der skal udarbejdes en kravspecifikation til godkendelse inden projektstart. *Bgl/12*

[2]

[2]https://dl.dropboxusercontent.com/u/91435/tlog/case_2.png

3 Case analyse

3.1 Funktionalitet

Monitoreringssystemet skal have mulighed for at modtage Syslog-beskeder og gemme dem i en database. Ved brug af filtre, der kan oprettes via et web interface, skal det være muligt at specificere, om man finder en bestemt type log for relevant eller irrelevant. Der vil være mulighed for at bede om at modtage en notifikation i tilfælde af, at et angivet event opstår. For at give systemet en vis form for adaptivitet, vil systemet benytte en modtaget log beskeds hyppighed til et early warnings system og beregne en score-prioritet, der kan sorteres efter.

3.2 Stabilitet

For at forsøge at undgå et single point of failure, skal det være muligt at starte flere instanser på andre servere.

3.3 Dokumentation

For at sikre den fremtidige brug og mulighed for videreudvikling af produktet skal der udarbejdes dokumentation af hele systemet.

4 Problemformulering

Der skal udvikles et monitoreringssystem, som hurtigt og sikkert skal kunne sende en notifikation til en eller flere ansvarlige.

- **Hvordan kan man bedst muligt sende en notifikation der gør den ansvarlige opmærksom på problemet?**

For at undgå irrelevant data, skal det være muligt at oprette et eller flere filtre, der specificerer, hvilke log-beskeder man ønsker at modtage.

- **Hvordan filtreres modtagne log-beskeder?**

Der ønskes at implementere en vis adaptivitet i filtreringsprocessen.

- **Hvordan kan man få en log-beskeds hyppighed til at få indflydelse på prioriteten og derved skabe en adaptivitet?**

For at sikre, at monitoreringssystemet kører, skal der implementeres en watchdog-funktion, der kan give en ansvarlig besked, hvis systemet stopper med at virke.

- **Hvordan skal watchdog-funktionen implementeres?**

Der skal være let adgang til at se og søge i modtagne log-beskeder via en web-browser.

- **Hvordan kan man udvikle et overskueligt layout?**

Systemet skal kunne håndtere et ubestemt antal klienter, der sender log-beskeder til systemet.

- **Hvordan kan systemet bedst muligt skaleres?**

5 Estimeret tidsplan

Opgave	Timer
Design database	8
Lav en Syslog parser	16
Lav et standardformat til at håndtere log-beskeder	4
Lave en funktion hvormed der kan gemmes log-beskeder	10
Lav en standardfunktion til at sende notifikationer	10
Lav et filterformat	8
Lav et filter-tjekker	8
Lav et early warning system	16
Lav et web-interface	30
Find en søgemaskine til log-beskeder	4
Lav en måde at sende log-beskeder til søgemaskinen	8
Lav et web-interface til søgemaskinen	8
Rapportskrivning	30

6 Metode og teknologivalg

Jeg har valgt at udvikle TLog i Python. Python er platformsuafhængigt, og jeg mener derfor, at det er det rette valg til dette projekt. Performancemæssigt er Python på ingen måde det hurtigste sprog, fordi koden bliver fortolket ved runtime, i stedet for at være kompileret som f.eks. C kode er.[3] For dette projekt mener jeg dog ikke, at det er et problem, da det der vil tage længst tid, er at skrive til databasen, hvilket er ude af Pythons hænder. Hvis der er kode, hvor det giver mening at optimere det i C eller C++, er det meget nemt at inkludere i Python koden.[4]

[3] http://stackoverflow.com/questions/3033329/why-are-python-programs-often-slower-than-the-equivalent-program-written-in-c-or

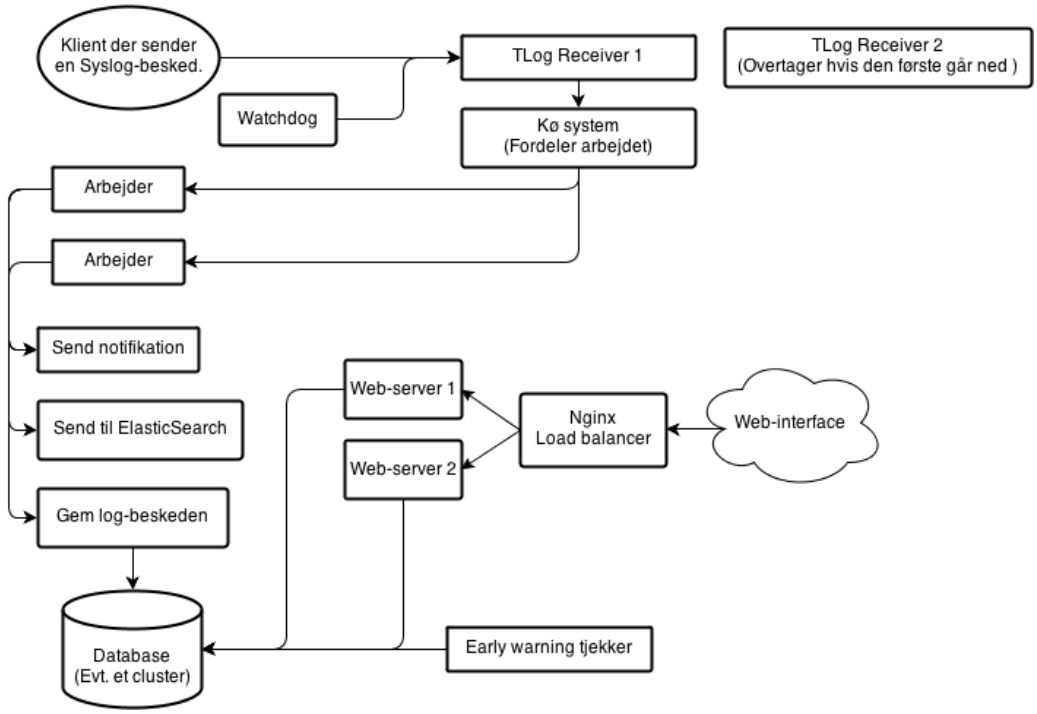
[4] http://docs.python.org/2/extending/extending.html

Til versionering har jeg valgt, at benytte Dropbox. Havde der været andre end mig, ville jeg have benyttet Git[5] i stedet, for at gøre det nemmere at arbejde sammen, i den samme kodebase.

For at holde styr på hvad jeg mangler, hvad jeg har udført og for at have en agil arbejdsproces, har jeg valgt at benytte Trello[6].

7 Topologi

Nedenstående giver et hurtigt indblik i hvordan jeg har tænkt, at systemet skal udvikles for at være skalerbart. Resten af rapporten giver indblik i de valg jeg har taget.



[7]

[5] http://git-scm.com/
[6] https://trello.com/
[7] https://dl.dropboxusercontent.com/u/91435/tlog/tlog_topologi.png

8 Log format

Der er brug for en eller flere standarder til at sende beskeder til TLog. I den første version ønsker jeg at understøtte et log format, der allerede er udbredt. Dette vil gøre implementeringen af TLog, i et allerede kørende miljø, væsentligt nemmere, da der ikke skal skrives klienter til de enheder (routere, switche, servere, etc.) der skal sende beskeder til TLog.

Syslog er en standard inden for computer-logging. Log-beskeder bliver sendt fra netværksenheder til en central Syslog-server.[8] Grundet alderen på Syslog og dens simple standard, er den ofte implementeret i de netværksenheder, man kan tænke sig at modtage log-beskeder fra. Dette er ideelt for den første version af TLog på grund af, at der generelt ikke vil være behov for at skrive klient-softwaren til netværksenhederne. Netværksenheder med en Syslog klient installeret skal blot konfigureres til at sende deres log-beskeder direkte til TLog serveren.

Der findes to Syslog standarder: RFC 3164[9] og dens stedfortræder RFC 5424[10] (Obs: der er andre "Syslog standarder", de beskriver dog kun ting som Syslog over TCP og kryptering).

RFC 3164 bliver i de fleste tilfælde benyttet på grund af, at den er væsentligt ældre end RFC 5424. Derfor er det nødvendigt, at TLog skal kunne understøtte begge standarder.

Begge standarder sender/modtager beskeder over UDP.

Ifølge de to standarder, bliver en besked markeret med en facility kode (auth, authpriv, daemon, cron, ftp, lpr, kern, mail, news, syslog, user, uucp, local0, local7) der indikerer, hvilken type software der sendte beskeden[11], og en severity kode (Emergency, Alert, Critical, Error, Warning, Notice, Info, Debug) der indikerer, hvor alvorligt beskeden skal tages[12]. Facility og severity koden bliver brugt til at beregne beskedens prioritetskode. Prioritetskoden bliver angivet som det første i Syslog-beskeden, indikeret af: "<[prioritets kode]>"" (se eksemplerne nedenfor).

[8] http://en.wikipedia.org/wiki/Syslog
[9] http://tools.ietf.org/html/rfc3164
[10] http://tools.ietf.org/html/rfc5424
[11] http://en.wikipedia.org/wiki/Syslog#Facility_Levels
[12] http://en.wikipedia.org/wiki/Syslog#Severity_levels

RFC 3164 log format:

```
<34>Oct 11 22:14:15 mymachine su: su root failed for
lonvick on /dev/pts/8
```

RFC 5424 log format:

```
<34>1 2013-08-11T22:14:15.003Z mymachine.example.com su - ID47 -
BOM\'su root\' failed for lonvick on /dev/pts/8
```

Ved at kigge på de to overstående eksempler kan jeg konstatere, at RFC 5424 tilføjer et versionsnummer, et andet tidsformat, en anden måde at indikere app navnet på, og hvad folkene bag RFC 5424 kalder for structured-data[13].

Structured-data er lidt interessant, da det giver mulighed for at sende struktureret data med i Syslog-beskeden, som Syslog serverne kan lære at forstå. Jeg har valgt at lære TLog at forstå structured-data, da jeg mener, at det er smart at kunne filtrere i det, sammen med det andet information, Syslog-beskeden kommer med.

9 Database

En type database: NoSQL, ville være interessant at prøve. Med "NoSQL" menes der generelt en key-value store database, men det betyder blot "ikke en SQL database". I key-value store databaser er tabeller ikke foruddefineret, som de er i SQL databaser. NoSQL databaser som mongoDB og RethinkDB understøtter f.eks. automatisk sharding. Dette vil være nødvendigt, hvis man vil fordele ens data ud på mere end 1 server.[14]

Grundet min personlige erfaring vil jeg fokusere på SQL databaser i den første version af TLog. Jeg har ingen erfaringer med NoSQL databaser og ved, at en SQL database, som MySQL, kan klare opgaven i langt de fleste tilfælde.

Jeg mener, at det ikke er op til mig at bestemme, hvilken database der skal benyttes. Om det er MySQL eller PostgreSQL, skal TLog være ligeglad med. Jeg ønsker at understøtte så mange SQL-databaser som muligt.

[13] http://tools.ietf.org/html/rfc5424#section-6.3
[14] http://en.wikipedia.org/wiki/NoSQL

Måden, jeg vil understøtte flere SQL implementeringer på, er ved at bruge SQLAlchemy til at håndtere alle SQL forespørgslerne. SQLAlchemy er et Object-relational mapping (ORM) værktøj, skrevet i Python, og det understøtter de fleste populære SQL databaser (MySQL, PostgreSQL, Microsoft SQL Server). Alle SQL forespørgsler bliver skrevet i Python og oversat til SQL (der bliver tilpasset alt efter, hvilken type SQL database, der er forbindelse til), af SQLAlchemy. Fordelen, ved at bruge SQLAlchemy, er den tid, det sparer mig at være fri for at skrive og teste SQL forespørgsler på de SQL databaser, der skal understøttes. Ulempen ved at benytte SQLAlchemy er, at det er langsommere end håndskreven SQL (til en bestemt type database), fordi der er et ekstra oversættelseslag.

9.1 Layout

Log-beskeder kunne blot blive gemt i en tabel, og så kunne systemet vise den tabel frem via et (eller andet) interface. Dette mener jeg ikke er særligt smart, da en liste over de senest modtagne log-beskeder er ganske uinteressant at kigge på, fordi at interessante log-beskeder nemt, vil kunne gemme sig i mængden af uinteressante log-beskeder, og kan derfor nemt blive overset.

Jeg mener, man kan være med til at skabe en bedre og mere overskuelig liste over modtagne log-beskeder ved at gruppere log-beskederne ud fra deres besked hash og ved at have relationer til forskellige tællere, der kan vise, hvor mange gange den samme besked er blevet modtaget, og hvor mange gange den er blevet modtaget i løbet af f.eks. 1 dag.

Selve log-beskeden skal dog også gemmes, da den indeholder andet information end blot en besked, som f.eks. et host-navn. Der skal derfor være en tabel til log-grupper og en til log-gruppernes log-beskeder.

Hvordan skal log-beskeden gemmes? Tabellen kunne defineres specifikt til at understøtte Syslog-beskeder. Problemet ved dette er, at det ville være meget besværligt at understøtte andre typer log-beskeder, senere hen.

For at kunne understøtte flere typer log-beskeder, har jeg kigget på, hvad en log besked generelt indeholder:

- Timestamp
- Hvor alvorligt en log-besked skal tages
- En besked tekst

Hvordan kan andre informationer gemmes, sammen med log-beskeden? Der kunne laves en tabel, der fungerer som en "`key value store`". Problemet ved dette er, at det kan giver en masse ekstra `inserts` til databasen. Desuden kan de `joins`, der skal laves i SQL forespørgselen, for at få fat i dataene, give dårlig performance, fordi det giver databasen mere at lave.

Jeg mener at være kommet på en bedre måde at gemme "andet log-data", sammen med log-beskeden ved, at benytte JSON (JSON er et menneskelæseligt struktureret format[15]). Jeg har valgt at benytte JSON fordi det er utroligt simpelt at arbejde med i Python. Jeg kunne have benyttet XML, men det er jeg ikke meget for og desuden mener jeg at det vil være overkill, fordi XML er designet som et "document markup language" som b.la.: bliver benyttet i `docx` filformatet.[16]

Den JSON formaterede tekst vil blive gemt i et felt kaldet `data`. Ved at gøre det på denne måde kan der dynamisk gemmes alt det data, man kan tænke sig, uden at det kræver flere `inserts`, da dataene bliver indsat i et enkelt `insert`. Måden passer også godt sammen med at gemme "structured-data", specificeret i RFC 5424, fordi den er dynamisk.

Det gør det også nemmere at tilføje data, som der ikke var tænkt på, senere hen, uden at skulle til at ændre i database-tabellerne. Desto mindre der skal ændres i database-tabellerne, desto bedre, fordi det kan tage meget lang tid at udføre en `alter` kommando på en stor tabel, fordi ændringen kan betyde at der skal ændres noget ville alle rækkerne (`rows`) der er gemt.[17]

Andet log data feltet, `data`, kan fylde unødvendigt meget. For at spare plads komprimeres dataene med zlib og bliver derefter encoded med base64 for at gøre data-overførslen nemmere. Jeg har valgt zlib, fordi den er implementeret i mange programmeringssprog (heriblandt Python) og bliver ofte benyttet i HTTP sammenhæng, for at opnå en mindre dataoverførsel. Ved at komprimere dataene i feltet udelukkes muligheden for at søge direkte i teksten med f.eks. MySQL's full text search. Jeg løser dette problem ved at benytte et bedre system til at søge i beskederne med, se Søgning.

[15] http://en.wikipedia.org/wiki/JSON

[16] http://stackoverflow.com/questions/4812440/proper-use-of-xml

[17] http://www.mysqlperformanceblog.com/2006/06/09/why-mysql-could-be-slow-with-large-tables/

For bedre, at forstå, hvordan "andet log data" bliver gemt, har jeg lavet et eksempel:

```
{
    "priority": 129,
    "procid": 123,
    "severity": 1,
    "facility": 16,
    "timestamp": "2013-03-12T12:01:02Z",
    "message": "alert_multiple_requests: ACTIVE",
    "app-name": "snort"
}
```

Som det kan ses i overstående eksempel, indgår beskeden (`message`) i andet data feltet: `data`. Jeg har valgt at gemme selve beskeden deri, da det ofte er den tekst, der vil fylde mest og vil blive komprimeret på grund af det.

9.2 Skalering

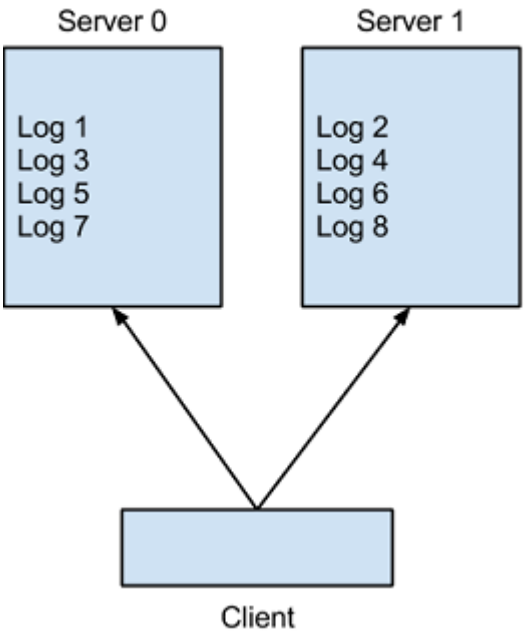
9.2.1 Lodret skalering

Lodret skalering er den simple løsning og skal forstås ved at give serveren hurtigere hardware (HDD, CPU, etc.). Dette vil i langt de fleste tilfælde være tilfredsstillende. Jeg har ikke konkrete tal på, hvor mange skrivninger et bestemt server setup kan håndtere.

Der er så mange variabler i at skulle lave en performance-test for TLog, at jeg ikke mener, at resultatet vil være brugbart, da man skal have en identisk opsætningen af systemet og hardwaren, for at kunne få et lignende resultat.

9.2.2 Vandret skalering

Ved vandret skalering benytter man en teknik kaldet sharding.



[18]

Sharding er normalt den sidste løsning man vil gå efter, hvis en opgradering af serveren (Lodret skalering) ikke er tilstrækkelig.[19]

For at understøtte sharding skal TLog have en måde, hvorpå den får at vide, hvilken database-server den skal sende til. Hvis der vælges en "tilfældig" database-server at sende til, skal den have en lookup database for at kunne finde log-beskeden igen. Dette er ikke smart og vil blot fordoble antallet af skrivninger.

En bedre måde er at generere et unikt id med universally unique identifier (UUID), konvertere det unikke id til et tal med crc32[20] og derefter dividere tallet med antallet af servere og bruge restværdien som serverens id. Ved at have det unikke id, vil det altid være muligt at beregne det samme server id så længe, at antallet af servere er uændret.

Hvad sker der, hvis der tilføjes en ekstra server? Det unikke id vil ikke længere blive beregnet om til det korrekte server id. Derfor skal der beregnes et nyt server id for alle gemte log-beskeder (ud fra det unikke id), og derefter skal de flyttes til den korrekte server. Her opstår der et dilemma: Skal hele log-systemet stoppes, mens log-beskederne bliver flyttet rundt, eller skal systemet lade nye log-beskeder

[18] https://dl.dropboxusercontent.com/u/91435/tlog/sharding_1.png
[19] http://en.wikipedia.org/wiki/Shard_(database_architecture)
[20] http://en.wikipedia.org/wiki/Cyclic_redundancy_check

blive tilføjet, mens log-beskederne bliver flyttet?

Jeg ønsker ikke at bruge tid på at implementere en sharding løsning, da jeg mener, at lodret skalering vil være tilstrækkeligt i langt de fleste tilfælde. Jeg mener også, at sharding skal være implementeret i databasen, som det f.eks. er i MySQL Cluster.

10 Beskedmodtagelse

Når TLog modtager en besked, skal den have en måde, ved hjælp af hvilken den kan identificere, hvilket format log-beskeden er formateret i. Er det en Syslog besked? Er den formateret efter RFC 3164 eller 5424?

10.1 Besked parsing

For at finde, hvilket format beskeden er sendt i, benytter jeg en Regular expression. En Regular expression er for det meste en hurtig og simpel måde at validere et stykke tekst og at trække relevante data ud med. Jeg kender ikke til en bedre løsning, der er lige så hurtig og dynamisk at skrive.

For hvert log-format, der skal understøttes, skal der skrives en Regular expression. På den måde er det nemt at finde ud af, hvilket log-format den modtagne log besked er formateret i, og om det er validt data, der er blevet modtaget.

Hvis en log-besked matcher en af de registrerede Regular expressions, kan det det relevante data trækkes ud. Jeg har lavet det, så en parser, som f.eks. til Syslog, skal returnere et object, som jeg har valgt at kalde for `Parsed`. Jeg har valgt at gøre det på denne måde, for at standardisere måden resten af systemet arbejder med log-beskeden på.

`Parsed` objectet består af en række felter (`hostname`, `level`, `data`, `standard`), som forventes at være til stede og udfyldt. Feltet `data` er beregnet til at være dynamisk. Dette betyder, at man kan tilføje x antal ubestemte felter. Dette er smart, fordi der findes mange forskellige typer log-formater, som også sender meget forskelligt data.

Ved structured-data (beskrevet i RFC 5424 standarden af Syslog protokollen) har jeg valgt ikke at benytte en Regular expression, grundet at structured-data's opbygning ikke passer til en Regular expression. I stedet har jeg valgt at gå bogstav for bogstav igennem og parse teksten på den måde. Jeg benytter forskellige variabler til at fortælle systemet, om den er i et felt eller i felts tekst.

10.2 Gruppering

For at give et bedre overblik over modtagne log-beskeder, grupperes log-beskederne med en beregnende hashværdi af besked teksten. Dette giver mulighed for at indføre stikprøver, early warning, grafer og andet historisk/statistisk information.

Jeg har valgt kun at gruppere beskedteksten, for ikke at få flere grupper der har den samme besked tekst.

For at give et overblik over, hvilke servere der har sendt log-beskeden, er der en tæller, der tæller, hvor mange gange en log-besked er blevet sendt fra en server.

Den enkelte log-besked bliver stadig gemt, da den kan indeholde mere specifikt information, som host-navn, etc.

10.3 Stikprøver

Ved at gruppere og tælle antallet af modtagne log-beskeder, mener jeg, at det ikke er nødvendigt at gemme hver eneste log-besked. For at finde ud af om log-beskeden skal gemmes, skal der laves en metode til dette.

Jeg har valgt, at man i konfigurationsfilen kan definere en liste, der, ud fra antallet af modtagne log-beskeder, returnerer en rate. Raten bliver benyttet til at fortælle, hvor ofte beskeden skal gemmes. Ved at dividere antallet af modtagne log-beskeder med raten, får man en restværdi. Er restværdien 0, skal log-beskeden gemmes.

Tid er også en faktor, der kan spille ind. På samme måde som ovenstående, kan man, i stedet for at specificere antallet af modtagne log-beskeder, specificere antallet af sekunder siden en tilsvarende log-besked sidst blev set.

Jeg mener, at dette er smart, dels fordi det sparer plads, men også for performance, da der ikke skal skrives lige så meget til databasen.

10.4 Filtrering

For at log-beskeder kan filtreres, skal der være en måde at definere et eller flere filtre. Dette kan gøres ved at definere faste felter i en tabel, hvor feltet indeholder en måde at matche teksten på. I mange tilfælde ville dette være en brugbar måde. Det er det bare ikke til TLog, på grund af det dynamiske felt `data` (beskrevet i Layout).

Derfor skal der være en dynamisk måde, at specificere, hvilken tekst et felt skal indeholde.

I Python er der en variabeltype kaldet dict. Dict ligner JSON til forveksling og er lige så dynamisk og nem at konvertere frem og tilbage fra. Eftersom feltet `data` indeholder JSON, er det ideelt at bygge filtret på samme måde.

Ved at kunne konvertere både filtret og log-beskeden til et `dict` object, vil det være meget nemt at skrive en metode, der systematisk går felterne igennem i filtret, og matcher feltet mod det samme felt i log-beskeden. For at validere, at feltet indeholder det "korrekte data", benytter jeg endnu engang Regular expression, da jeg mener at det er den simpleste måde at kunne validere, hvad kan være komplekst data. Det vil være smart at have mulighed for at definere mere end en Regular expression per felt. Derfor har jeg valgt at understøtte dette, da man kan være interesseret i at det samme filter kan matche flere forskellige typer af log-beskeder.

I et filter er man normalt interesseret i at finde tekst, der matcher, men man kan også være interesseret i at kunne specificere, hvad filtret ikke skal matche. Derfor vil jeg understøtte, at man skal kunne specificere, hvad den skal og ikke skal matche for at være valid i forhold til filtret.

Et filter er der ikke meget ved, hvis man ikke kan specificere, hvad der skal ske med de log-beskeder, det matcher. Jeg har lavet det, så man kan specificere, om log-beskeden skal "gemmes", om der skal sendes en "notifikation" og/eller om log-beskeden skal tilføjes til "søgemaskinen".

Log-beskeden er der ikke meget ved, hvis den ikke bliver gemt. Derfor har jeg valgt at gem-indstillingen overtrumfer notifikationen. Dette betyder, at hvis man vælger ikke at gemme beskeden, vil man heller ikke modtage en notifikation.

For at give en bedre forståelse, for hvordan et filter kan se ud, har jeg lavet et eksempel:

```
{
    "match": {
        "hostname": [
            "^web-[0-9]+.*$"
        ]
    },
    "not_match": {
        "data": {
```

```
            "severity": [
                "[4-8]+"
            ]
        }
    },
    "store": true,
    "notify": true,
    "searchable": true
}
```

Eksemplet ovenfor er et filter, der matcher log-beskeder, der bliver sendt fra et `hostname`, der starter med `web-` og ikke har et `severity` level mellem 4 og 8. For hvert defineret felt i filtret, er der mindst et Regular expression, der skal matche det samme felt i log-beskeden, ellers vil det blive bestemt, at filtret ikke stemmer overens med log-beskeden, dette giver mulighed for at flere forskellige log-beskeder kan matche det samme filter. Som det også kan ses i ovenstående eksempel, vil de log-beskeder, der matcher, blive gemt; en notifikation vil blive sendt til de ansvarlige, og den vil blive sendt til søgemaskinen.

Hvis der er to eller flere filtre, der matcher den samme besked, kan der opstå en konflikt. Et filter mener, at beskeden skal gemmes, og et andet mener det modsatte. Her har jeg valgt, at alle filtrene skal være enige om, at log-beskeden ikke skal gemmes, for at dette vil ske, da jeg menere at man ellers nemt kan komme til udelukke en log-besked, uden at være klar over det. Jeg har lavet det, så når en log besked bliver gemt, bliver der oprettet relationer til de filtre, der sagde go for den handling. Det smarte ved dette er, at det er let at se, hvilke filtre man skal redigere, hvis man ikke er tilfreds med resultatet.

10.5 Skalering

Det mest resursekrævende i TLog sker efter modtagelsen af en log-besked. Hvis arbejdet ikke bliver fordelt, kan man ende op med at "tabe" beskeder, hvis der modtages mere, end hvad processen kan nå at validere. Grunden til dette er, at beskedmodtagelsesprocessen er single threaded[21], hvilket betyder, at den kun kan udføre én log-besked ad gangen. Den modtager beskeden fra en åben UDP socket. Denne UDP socket har en begrænset bufferstørrelse, og når den er fuld, vil den ignorere,

[21] http://en.wikipedia.org/wiki/Single_threading

hvad den modtager af nyt data, indtil der er plads igen. Bufferstørrelsen kan udvides. Dette er dog ikke skalerbart, og går serverne ned, vil alt data i bufferen være tabt.

For at formindske sandsynligheden for at tabe beskeder, gør jeg det muligt at fordele arbejdet ud på flere processer/servere. Fordeling foregår ved, at sende alle log-beskederne videre til et kø-system. På samme måde, som ved SQL database valget, ønsker jeg også her, at det ikke er op til mig at vælge, hvilket kø-system der skal benyttes.

For at opnå det frie valg benytter jeg Celery. Celery giver et standard interface til forskellige kø-systemer, på samme måde som SQLAlchemy giver et standard interface til forskellige SQL databaser. Ved at benytte Celery kan der benyttes kø-systemer, som RabbitMQ og en bunke andre. Et kø-system, som RabbitMQ, er skrevet i Erlang[22], det er open source, har understøttelse for fail over og kan skaleres vertikalt.[23]

Jeg har valgt at understøtte besked-parsing med og uden et kø-system, fordi det ikke vil være alle, der modtager nok log-beskeder til at have brug for at fordele dem ud på flere processorer/servere. Ved ikke at kræve, at der skal sættes et kø-system op, forenkles systemet og giver mulighed for en hurtigere opsættelse. Dermed kan du f.eks. teste, om systemet er noget for dig uden at skulle til at sætte et 3. system op.

11 Team

Der er brug for en nem måde at organisere brugere på. Derfor har jeg valgt at gøre det muligt at fordele dem på teams.

Jeg mener, det gør systemet nemmere at administrere, hvem der skal være medlem af hvilke filtre hvis brugerne er fordelt i teams. F.eks. kan der være et team, der hedder "Web udviklere", som er medlem af et eller flere filtre, der vil matche webrelaterede beskeder. Et andet team kunne være "Infrastruktur", der f.eks. kun vil få notifikationer, der omhandler netværket. Når teams bliver medlemmer af filtre, er det nemmere blot at tilføje en ny bruger til et team, og så er brugeren medlem af de korrekte filtre.

[22] http://www.erlang.org/
[23] http://www.rabbitmq.com/

12 Notifikationer

Notifikationer er en vigtig del af TLog. Det er ikke altid, at man aktivt følger med i, hvad der sker på web-interfacet. Det kan være, at man sover eller blot laver noget andet. Derfor skal systemet have mulighed for at sende notifikationer.

Hvordan kan systemet gøre den ansvarlige opmærksom på et problem, der er opstået? Og helst inden, at brugerne når at klage, over at internettet f.eks. "ikke virker".

En enhed, de fleste af os har ved hånden, 24 timer i døgnet, som kan gøre opmærksom på sig selv, er vores smartphones. Der er standardmåder at kommunikere med smartphones på som: SMS og email. Andre metoder er applikationer, der kan installeres på ens smartphone, som f.eks: Pushover. Pushover påstår, at de repræsenterer en nem måde at sende real-time notifikationer på (hvilket har vist sig at være sandt).[24]

Jeg mener ikke, at det er op til mig at bestemme, hvilken måde man skal modtage notifikationerne på. Derfor vil jeg gøre det muligt at implementere alle de forskellige måder, man kan komme i tanke om. Hvis der skal understøttes flere forskellige måder at sende notifikationer på, skal der være en standard måde, at sende en notifikation på i programmet. Dette kan gøres ved, at udvikle et plugin system. Jeg har dog valgt, at det vil blive i en senere version af TLog.

Problemet, der opstår nu, er: Hvor skal man bestemme, hvilken type notifikation der skal sendes? En måde vil være at definere det i filtret. Her kunne man gøre det muligt at definere, hvilken måde alle skal modtage notifikationer på. Jeg ser et problem ved denne måde. For at det kan virke skal alle være enige, om hvordan de vil modtage deres notifikationer.

Mennesker er forskellige, nogle vil måske helst modtage notifikationerne over email, mens andre ønsker det på en anden måde. Derfor vil jeg gøre det, så brugeren, via sin personlige indstillingsside, selv kan specificere, hvordan de ønsker at modtage notifikationerne.

Eftersom det er filtret, der vil være triggeren for at sende notifikationen, har jeg gjort, så man kan slå notifikationer fra i filtret. Jeg mener, dette er vigtigt, fordi det ikke er altid, at man ønsker at modtage en notifikation, bare fordi, der er en besked, der matcher et filter.

[24] https://pushover.net/

For at kunne præsentere brugeren for en liste over notifikations muligheder, er der oprettet en konstant i systemet. Denne konstant indeholder de forskellige notifikations typer.

Der skal også være et sted at gemme brugerens valg. En måde er at gemme dette i en relations-tabel. En anden måde er at gemme valgene, som JSON formateret tekst, i et felt i brugertabellen. Jeg mener, at det er en bedre løsning at gemme brugeres valg i et tekstfelt, fordi det vil spare systemet for at skulle lave en ekstra forespørgsel.

13 Adaptivitet

For at opnå en vis form for adaptivitet har jeg kigget på 3 forskellige måder.

Den første er en ide om, hvordan beskedens vigtigheds (severity) niveau, plus antallet af modtagne log-beskeder, kan generere en ny form for vigtigheds-niveau, der har et ekstra niveau. Først vendes vigtigheds-niveauerne på hovedet, så 0 bliver 7 osv. Hvis vigtigheds-niveauet er 3, og log-beskeden er blevet modtaget 7 gange inden for et bestemt tidsinterval, vil dens vigtigheds-niveau blive ændret til 3,7. Er den modtaget 10 gange, vil den stige et helt vigtigheds-niveau: 4,0. Problemet, jeg ser ved denne måde, er, at den ikke skalerer. Det er ofte, at informationsniveauet f.eks. bliver sendt, oftere end fejl-niveauet.

Den anden måde er, hvad jeg kalder for "score". Log-gruppen får beregnet en score, ved at beregne den naturlige logaritme af antallet af modtagne log-beskeder for log-gruppen, gange det med 600 (for at tallet giver en større effekt) og til sidste addere tallet med det nuværende "timestamp".

```
score = math.log(times_seen) * 600 + time.time()
```

Jeg valgte at bruge den naturlige logaritme for at få det til at skalere bedre, så stigningen af antallet af modtagne log-beskeder bliver gjort mere "jævn". F.eks. vil tallet 5, beregnet med den naturlige logaritme, være: 1.60943 og ved 50 vil den være: 3.91202. Dette giver en forskel på 2,30258 i forhold til 45. Den beregnede score kan herefter benyttes til at sortere efter. Ved hjælp af denne måde giver log-gruppens antal af modtagne log-beskeder en højere plads på listen, og der skabes derved en adaptivitet. Ved at have tilføjet et timestamp, vil log-gruppen automatisk falde i pladser, til nyere log-grupper.

Den tredje måde er beskrevet i Early warning.

13.1 Early warning

Early warning kan i visse tilfælde blive detekteret ved at opdage pludselige udsving i antallet af modtagne log-beskeder, inden for et bestemt tidsinterval.

Log-beskeder kommer med deres egen information om, hvor vigtige de er, og denne information kan blive opfanget af et filter. Et tegn på en early warning kan opstå, ved de log-beskeder, der (hver for sig) ikke betyder noget særligt, tilsammen kan have en større betydning. Det kunne f.eks. være en skrivefejl til en disk, en fejl der i sig selv ikke behøver at prioriteres højt, men et usædvanligt antal inden for en kort periode kan betyde et større problem.

For hvert filter gemmes der, hvor mange log-beskeder der er modtaget i intervaller af 15 minutter.

En måde, jeg har overvejet at opdage early warnings på, er ved at tage gennemsnittet af intervallerne mellem de modtagne log-beskeder (inden for f.eks. 2 timer) og sammenligne det tal med antallet inden for de f.eks. seneste 15. minutter. Problemet ved blot at tage gennemsnittet er, at det ikke tager højde for afvigelsen mellem intervallerne.

Jeg mener, at der kan beregnes et bedre gennemsnitstal, der tager højde for afvigelser ved at tage gennemsnittet (Mean[25]) af intervallerne og addere det med medianen af den absolutte afvigelse (MAD[26]) i intervallerne, gange afvigelsen med 2 (for at give afvigelsen en større effekt på slutresultatet), og til sidst dividere tallet med det antal minutter (15), der benyttes til at opdele intervallerne i, for at normalisere tallet.

```
previous = mean + mad * 2 / normalize_minute
```

I filtret laver jeg det, så man kan specificere en stigningsprocent. Den skal være større end eller lig med, inden det vil udløse en notifikation. Jeg har valgt, at inden en notifikation kan blive sendt, skal der være data for de sidste 2 timer og 15 minutter, da jeg menere at det er nødvendigt, for at kunne beregne en mere præcis stignings procent.

Dette er blot en af sikkert mange måder, man kan udvikle et "early warnings system" på.

[25] http://en.wikipedia.org/wiki/Mean
[26] http://en.wikipedia.org/wiki/Median_absolute_deviation

14 Søgning

Søgning er en vigtig del af TLog, for at gøre det let at finde de log-beskeder man leder efter. På grund af at der kan være meget irrelevant data, skal det være muligt at søge meget specifikt, men også løst, efter log-beskeder.

Først har jeg kigget på at benytte de søgefunktioner, databasen understøtter. MySQL har f.eks. mulighed for "full-text" søgning i et tekstfelt. Problemet ved f.eks. MySQL er, at den kræver specifikke felter at søge i. Systemet har et meget "løst" forhold til felter (beskrevet i Database Layout) og gemmer de fleste af dataene (formateret som JSON) i et felt. Dette gør databasen ubrugelig, som søgemaskine. Jeg har derfor brug for en søgemaskine, der kan søge i dynamiske felter.

ElasticSearch er en fleksibel, distribueret, real-time søgemaskine[27]. Den passer perfekt til mit formål, og den er skalerbar. Det smarte ved ElasticSearch er bl.a., at den forstår JSON. Dette gør den meget nem at implementere i TLog, fordi at JSON, i forvejen bliver benyttet til at gemme dele at log-beskeden i.

ElasticSearch kommer indbygget med en søgefortolker, hvor man både kan søge løst og søge efter specifikke ting. Det er f.eks. nemt at fortælle den, hvad man ønsker, at et bestemt felt skal indeholde.

Jeg har valgt, at alle log-beskeder, der bliver modtaget og matcher et filter med `searchable` sat til sand, bliver sendt til indeksering i ElasticSearch. Her bliver der ikke taget stikprøver, som der bliver gjort ved log-gruppering, da jeg mener, at det er bedre at søge efter enkle log-beskeder. Fordi man normalt kun er interesseret i log-beskeder fra en bestemt server, eller andet specifikt information, som log-beskederne er blevet gemt med.

15 Interface

For at gøre TLog brugbar, skal der være et interface.

Interfacet kunne være en applikation, der bliver installeret lokalt på computeren. Problemet ved dette er, at man skal installere applikationen, og det vil være besværligt at lave en måde at linke til log-beskeder i systemet.

Da interfacet ikke har brug for noget på klienten (som I/O adgang), mener jeg, at en web applikation er en bedre løsning. Med en web applikation er det også let at

[27] http://www.elasticsearch.org/

lave den platformsuafhængig og at dele log-beskeder via en URL-adresse. Dette er smart, hvis en URL-adresse er med i en modtaget notifikation. Det gør det let og hurtigt at få mere information om problemet, hvis det ønskes.

Når man logger ind via web-interfacet, mener jeg, at det er smart, hvis man får en oversigt over, hvordan det står til i systemet. Med det mener jeg, at man får en graf i toppen af skærmen - en graf der viser antallet af modtagne log-beskeder fordelt over en uge. Under grafen kan der være en liste over de senest modtagne log-beskeder, inden for de seneste 24 timer.

For at man kan følge med i modtagne log-beskeder i "real-time", har jeg lavet, hvad jeg kalder for en "stream". "Stream"-siden indeholder en liste over grupperede modtagne log beskeder. For at vise brugeren relevante log-grupper, vises der kun log-grupper, der har matchet en eller flere af de filtre brugeren er medlem af. Det vil være smart at kunne sortere de viste log-grupper, for at kunne give en andet syn på hvad der er modtaget af log-beskeder. Det kunne f.eks. være log-gruppens vigtigheds-niveau. Det kan også være at man kun er interesseret i log-grupper der har matchet et bestemt filter. Derfor har jeg gjort det muligt at ændre sorteringens typen, filtrer efter et bestemt filter og/eller en log-gruppes status (unresolved/resolved). For at få mere information om en log-gruppe og se de log-beskeder, den indeholder, har jeg gjort det muligt at klikke på log-gruppen, på "stream"-siden, der vil sende en videre.

Datoer på web-interfacet fremtræder ikke i et normalt datoformat. Jeg har i stedet valgt, at der bliver skrevet antallet af minutter/timer/dage på siden. Dette mener jeg er bedre, da man ikke selv skal bruge tid på at beregne det. Det er stadig muligt at se den fysiske dato ved at holde musen over. "2013-09-10T16:27:00" kan f.eks. blive formateret til "3 minutter siden". Da man ofte kan have siden stående i længere tid, har jeg lavet det, så tid-siden automatisk bliver opdateret. Dette betyder, at hvis der står 10 minutter siden, er det 10 minutter siden.

Søgeinterfacet har jeg forsøgt at lavet meget simpelt. Siden indeholder blot et input felt, hvor man kan indtaste sin søgeforespørgsel. Det ville også være smart at have en måde at kunne søge inden for bestemte datoer. Dette har jeg dog valgt ikke at understøtte i den denne version af TLog. Jeg har valgt at søgeresultaterne er de enkelte log-beskeder, og dermed er de ikke grupperet, som de er på stream-siden. Dette mener jeg er bedre, fordi man ofte ikke har brug for den grupperede version, men i stedet de enkelte log-beskeder, når man søger. F.eks. kan det være at, man kun er interesseret i log-beskeder fra en bestemt server, eller andet specifikt information, som log-beskederne er blevet gemt med. Jeg har dog lavet det, så det er

muligt at klikke på selve log-teksten for at gå til log-gruppen, hvis log-beskeden er medlem af en log-gruppe.

16 Watchdog

TLog er ikke meget ved, hvis det er nede. Hvordan får man en notifikation om, at systemet er nede, når systemet der skal sende notifikationen, er det system, der er nede?

For at løse den udfordring indfører jeg en form for "heartbeat". Fordi hele systemet kan være fordelt ud på flere servere, er det ikke nok blot at "pinge" en server. At "pinge" en server giver en falsk tryghed om at systemet virker. En ping, tager ikke højde for om f.eks. at harddisken er gået i stykker eller at der er en bug, der gør at log-beskederne ikke bliver gemt.

For at teste bedst muligt om systemet kører, har jeg valgt at sende en "heartbeat request" til TLog, som var det en almindelig log-besked. Når Watchdog-systemet sender beskeden, og TLog modtager og genkender denne log-besked (som værende en forespørgsel for et "heartbeat"), sætter den det nuværende timestamp i en tabel, i databasen. Watchdog-systemet tjekker herefter, om "heartbeatet" er blevet sat inden for de seneste 3 minutter. Er "heartbeatet" ikke blevet sat, antager watchdogen, at systemet er nede og sender en notifikation til de ansvarlige, som kan specificeres i konfigurationsfilen.

Ved at gøre det på overstående måde tjekkes der, om receiveren, workeren og om databasen virker. Fordi "heartbeat requestet" går den samme vej som en almenlig log-besked.

Hvis det ikke er muligt for watchdog-systemet at sende notifikationen, fordi email-systemet måske også er nede, har jeg valgt at den skal blive ved, indtil notifikationen er sendt. Her kunne man forstille sig at email-systemet ville blive lavet, men at log-systemet ville blive overset, hvis ikke watchdogen forsøgte at blive ved med at sende notifikationen.

17 Test

For at skabe en mere stabil udviklingsproces, har jeg skrevet "unit tests", der tester det meste af koden. At have "unit tests" betyder, at jeg kan skrive store dele af koden om, køre testen og se, om det virker.

Det gør det også nemmere for andre at skrive videre på TLog uden at kende til anden kode, end hvad personen har skrevet om.

I TLog er der 50+ tests.

18 Realiseret tidsplan

Opgave	Estimeret timer	Realiseret timer
Design database	8	8
Lav en Syslog parser	16	8
Lav et standardformat til at håndtere log-beskeder	4	2
Lave en hvormed der kan gemmes log-beskeder	10	12
Lav en standardmåde til at sende notifikationer	10	10
Lav et filter format	8	4
Lav et filter tjekker	8	4
Lav et early warning system	16	11
Lav et web-interface	30	31
Find en søgemaskine til log-beskeder	4	1
Lav en måde at sende log-beskeder til søgemaskinen	8	2
Lav et web-interface til søgemaskinen	8	4
Rapportskrivning	30	40

19 Logbog

Mandag d. 19-08-2013

I dag har jeg haft et møde med min vejleder, Bo Guldstrand, hvor vi har talt om projektet. Jeg fik skrevet en start på case analysen, problemformuleringen og kravspecifikationen. Vi aftalte, at jeg skulle arbejde hjemmefra det meste af tiden, på grund af jeg er ene mand og har en lang køretur. Efter mødet tog jeg hjem og begyndte så småt at designe databasen.

Tirsdag d. 20-08-2013

I dag har jeg rettet på lidt ting i case analysen, problemformuleringen og kravspecifikationen. Venter på at få kommentarer fra Bo. Jeg fik færdiggjort første version af databasen. Der skal nok laves en del rettelser senere hen. Jeg fik skrevet en syslog parser og en metode til at validere en log-besked mod et filter.

Onsdag d. 21-08-2013

I dag har jeg skrevet bruger og team klasserne, rettet lidt i databasen og begyndt at skrive notifikationsmetoden.

Torsdag d. 22-08-2013

Notifikations metoden er gjort færdig, og jeg er begyndt på web-interfacet.

Fredag d. 23-08-2013

Arbejder fortsat på web-interfacet og er begyndt at kigge på en algoritme til et early warning system.

Mandag d. 26-08-2013

I dag har jeg arbejdet på early warning systemet og web-interfacet og har rettet nogle fejl i notifikations systemet.

Tirsdag d. 27-08-2013

I dag har jeg arbejdet på web-interfacet.

Onsdag d. 28-08-2013

I dag har jeg arbejdet på web-interfacet og tilføjet nogle grafer. Rettet nogle fejl i notifikationssystemet. Jeg har fået beskedmodtagelsen til at skalere med celery.

Torsdag d. 29-08-2013

I dag har jeg arbejdet på at implementere ElasticSearch til søgning i logs og tilføjet grafer, der viser antallet af modtagne beskeder for log-grupper og -filtre.

Fredag d. 30-08-2013

I dag har jeg fortsat arbejdet på web-interfacet, bl.a. ved at søge interfacet og har tilføjet en ny notifikations mulighed: pushover. Jeg fik skrevet email klassen om og gjort den mere standardiseret. Notifikations typerne er blevet flyttet fra databasen til en konstant i koden.

Mandag d. 02-09-2013

I dag har jeg skrevet procesrapport og lavet en måde at initialisere konfigurations-filen.

Tirsdag d. 03-09-2013

I dag har jeg skrevet procesrapport.

Onsdag d. 04-09-2013

I dag har jeg skrevet procesrapport.

Torsdag d. 05-09-2013

I dag har jeg skrevet produktrapport og lavet flow charts.

Fredag d. 06-09-2013

I dag har jeg skrevet produktrapport og dokumenteret ER-diagrammet for databasen.

Mandag d. 09-09-2013

I dag har jeg rettet i procesrapporten, og lavet en sign up funktion.

Tirsdag d. 10-09-2013

I dag har jeg rettet procesrapport, produktrapport og nogle bugs i TLog.

Onsdag d. 11-09-2013

I dag har jeg rettet procesrapport, produktrapport og nogle småfejl i web-interfacet.

Torsdag d. 12-09-2013

I dag har jeg rettet procesrapport, produktrapport og nogle fejl der opstod da jeg skulle installere TLog på Debian.

Fredag d. 13-09-2013

I dag har jeg rettet procesrapport, produktrapport og nogle fejl i web-interfacet.

20 Konklusion

Hvordan kan man bedst muligt sende en notifikation, der gør den ansvarlige opmærksom på problemet?

Jeg fandt frem til en kombination af email og Pushover. Det er op til brugerne selv, om de ønsker en email og/eller en besked via Pushover. På grund af, at man normalt modtager emails, kan man nemt overse alarmer. Med Pushover er det muligt at bede modtageren om at anerkende, at de har set beskeden, inden den stopper med at give lyd fra sig. Med dette mener jeg, at TLog kan tilbyde en ordenlig og sikrere måde, at sende notifikationer på i forhold til email.

Hvordan filtreres modtagne log-beskeder?

En modtaget log-besked bliver testet mod de filtre, der er blevet specificeret i TLog. Et filter bliver defineret i JSON, hvor det er muligt at specificere med en eller flere Regular expressions, hvad et felt skal indeholde (som f.eks. hostname, severity level, osv.). I filteret kan det bestemmes, om log-beskeden skal gemmes, sendes som notifikation, og om den skal gøres søgbar.

Hvordan kan man få en log beskeds hyppighed til at få indflydelse på prioriteten og derved skabe en adaptivitet?

Ved at tage den naturlige logaritme af antallet af modtagne log-beskeder, gange det med 600 og addere det med det nuværende timestamp, kan der beregnes en "score", som kan sorteres efter.

En anden måde, hvor log-beskedens hyppighed også bliver benyttet, er ved "Early warning" systemet. Her bliver der beregnet et gennemsnit over de sidste 2 timer, normaliseret til et minut og sammenlignet med det seneste minut. Overstiger forskellen i procent det specificerede antal i filtret, bliver der sendt en notifikation.

Hvordan skal watchdog funktionen implementeres?

Watchdogen er skrevet som et program, der en gang i minuttet sender en `TLOG_WATCHDOG` besked, over Syslog protokollen. Når TLog receiveren modtager `TLOG_WATCHDOG` beskeden, skriver den et timestamp i databasen. Dette timestamp bliver set som et "heartbeat". Watchdog programmet tjekker herefter, om der er skrevet et heartbeat i databasen, inden for de seneste 3 minutter. Er dette ikke tilfældet, sender den en notifikation til de ansvarlige, som er blevet specificeret i konfigurationsfilen. Når TLog er oppe at køre igen, sender den en informationsbesked til de ansvarlige om dette.

Hvordan kan man udvikle et overskueligt layout?

Ved at gruppere de modtagne log beskeder og præsentere dem i en overskuelig liste, der tydeligt viser, hvor alvorlig log gruppen skal tages, via et web interface. Ved også, at tilføje grafer der viser modtagne log beskeder, fordelte på bestemte tidsintervaller, giver et godt overblik over, hvornår log beskeden opstod.

Hvordan kan systemet bedst muligt skaleres og minimere single point of failure?

Der kan startes flere processer på flere servere af TLog receiveren, som modtager Syslog beskederne. Der kan bruges en load balancer, eller man kan blot bede klienterne om at sende til forskellige TLog receiveres.

Filtrering og alarmering kan sendes til et kø-system, som flere processer/servere kan hamstre fra. Celery bliver benyttet til at give et standard interface til flere forskellige kø-systemer.

Databasen kan skaleres ved f.eks. at benytte SSD's i et RAID eller ved at sætte et cluster op.

Single point of failure kan undgås ved at sætte 2 TLog receivers, 2 kø-systemer, 2 arbejdere til kø-systemet og 2 databaser med replikering, op.

21 Perspektivering

For at få systemet til at yde bedre, vil det være en god ide at få indført en cache foran databasen. Der er mange forespørgsler som: "hent filtre" der skal tjekkes og "eksisterer log-gruppen i forvejen", der kunne hentes hurtigere.

Af sikkerhedsmæssige årsager skal der laves en række rettigheder, som kan gives til brugere/teams. Det kan være et problem, at enhver kan oprette en bruger og tilmelde sig de teams, de har lyst til.

Det er ikke altid smart at sende data over UDP fra forskellige netværk. Det ville derfor være smart også at kunne modtage log-beskeder over HTTP/HTTPS.

Til søgeinterfacet vil det være smart at få implementeret en måde at søge inden for bestemte datoer.

For at gøre det nemmere at implementere nye former for notifikations typer, vil det være smart at få lavet et plugin-system. At skrive og teste et nyt filter kan være besværligt. Dette skal gøres bedre ved at gøre det muligt at køre filtret i en slags testmiljø, inden det ryger ud i produktion.